

- The hashing of a key to an array cell filled with a different key is called a collision.
- Collisions can be handled in two major ways: open addressing and separate chaining.
- In open addressing, data items that hash to a full array cell are placed in another cell in the array, chosen by following a prescribed probing sequence.
- In separate chaining, each array element consists of a linked list or binary tree. All data items hashing to a given array index are inserted in that chaining structure.
- We discussed three kinds of open addressing probe sequences: linear probing, quadratic probing, and double hashing.
- In linear probing the step size is always 1, so if x is the hash address calculated by the hash function, the probe goes to x , $x+1$, $x+2$, $x+3$, and so on.
- In linear probing, contiguous sequences of filled cells can appear. They are called primary clusters, and they reduce performance.
- In quadratic probing, the offset from the hash address, x , is the square of the step number, so the probe goes to x , $x+1$, $x+4$, $x+9$, $x+16$, and so on.
- Quadratic probing eliminates primary clustering but suffers from the less severe secondary clustering.
- Secondary clustering occurs because all the keys that hash to the same value follow the same sequence of steps during a probe.
- All keys that hash to the same value follow the same probe sequence in quadratic probing because the step size does not depend on the key.
- Quadratic probing only visits—or covers—about half the cells in the hash table.
- In double hashing, the step size depends on the key and is obtained from a secondary hash function.

- If the secondary hash function returns a value s in double hashing, the probe goes to $x, x+s, x+2s, x+3s, x+4s$, and so on, where s depends on the key but remains constant during the probe.
- The number of probed cells required to find a specified item is called the probe length.
- The load factor is the ratio of the number of data items stored in a hash table to the table size.
- The maximum load factor in open addressing should be around 0.5. For double hashing at this load factor, unsuccessful searches have an average probe length of 2.
- Search times go to infinity as load factors approach 1.0 in open addressing.
- It's crucial that an open-addressing hash table does not become too full.
- A load factor of 1.0 is appropriate for separate chaining.
- At load factor 1.0 a successful search in separate chaining has an average probe length of 1.5, and an unsuccessful search, 2.0.
- Probe lengths in separate chaining using lists increase linearly with load factor.
- Probe lengths in separate chaining using balanced binary trees increase logarithmically with load factor.
- By properly managing the load factor to limit probe lengths and collisions, hash tables have effectively $O(1)$ performance for search, insertion, and deletion of a single item.
- Traversing a hash table takes $O(N)$ time, where N is the maximum number of items inserted.
- Hash tables need $O(N)$ storage and take more space than a simple array takes to store N items.
- A string can be hashed by multiplying the numeric value of each character by a different power of a constant and adding the products.

- To avoid overflow with large numbers, you can apply the modulo operator at each step in the process, if a polynomial function and Horner's method is used.
- Hash table sizes should generally be prime numbers. Using prime numbers helps minimize the chance of collisions without knowing anything about the distribution of keys.
- When hash tables grow exponentially, the cost of inserting N items remains $O(N)$, even though many items must be reinserted at each doubling of the table.
- Hash tables can be used for external storage. One way to do this is to have the elements in the hash table contain disk-file block numbers. The blocks contain a limited number of records such that the load factor is kept low.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. Using Big O notation, how long does it take (ideally) to find an item in a hash table?
2. A(n) _____ transforms a range of key values into a (possibly large) number, which can be mapped to a range of index values.
3. The typical operation used to map large numeric ranges into small ones is _____.
4. When different keys map to the same index in a hash table, _____ occurs.
5. Open addressing refers to
 - a. keeping many of the cells in the array unoccupied.
 - b. using a parameter in the hashing function to expand the range of cells it can address.
 - c. probing at cell $x+1$, $x+2$, and so on, until an empty cell is found.

- d. looking for another location in the array when the original one is occupied.
6. Searching for a key by testing adjacent cells in the hash table is called _____.
 7. What are the first five offsets from the original address in quadratic probing?
 8. Secondary clustering occurs because
 - a. many keys hash to the same location.
 - b. the sequence of step lengths is always the same.
 - c. too many items with the same key are inserted.
 - d. the hash function maps keys into periodic groups.
 9. Double hashing
 - a. should use a different hash function than that used for the hash address and compute a step size from the hashed value.
 - b. applies the same hash function to the hash address, instead of the key, to get the next hash address.
 - c. is more effective for separate chaining than for open addressing.
 - d. decreases the search time by a factor of two.
 10. Separate chaining involves the use of a(n) _____ or _____ at each hash table cell.
 11. A reasonable load factor in separate chaining is _____.
 12. True or False: A possible hash function for strings involves multiplying each character value by a number raised to a power that increases with the character's position.
 13. The size of the hash table should _____ to minimize the number of collisions, in general.
 14. If digit folding is used in a hash function, the number of digits in each group should reflect _____.
 15. In which of the open address probing methods does an unsuccessful search take longer than a successful search?

16. In separate chaining with linked lists, the time to insert a new item
 - a. increases as the logarithm of the load factor.
 - b. is proportional to the ratio of items in the table to the number of table cells.
 - c. is proportional to the number of lists.
 - d. is proportional to the percentage of filled cells in the table.
17. When hash tables double or more than double in size when insertions exceed a threshold and the items must be rehashed into the array, the overall time taken to insert N items is
 - a. $O(\log N)$ time.
 - b. $O(N)$ time.
 - c. $O(N \times \log N)$ time.
 - d. $O(N^2)$ time.
18. Rank order these data structures for their “unused” memory in storing the exact same set of N items: a sorted linked list, an AVL tree, and an open addressing hash table using double hashing and a load factor of 0.6. Unused memory means cells or fields that are allocated but not filled (or filled with `None`) instead of a value or link to another structure.
19. True or False: In external hashing, it’s important that the blocks never become full.
20. In external hashing, all records with keys that hash to the same value are located in _____.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 11-A A person boarding a long train and looking for an empty seat and cars entering a highway are pretty good analogies to the way open addressing looks for an empty cell to hold an item. Can you think of real-world processes that act like separate chaining? Think of cases in which there is some initial choice about where people or things go, followed by hunting through a list based on the first choice to find their

final destination. When you think of one, how likely is it to have collisions? Can you think of ways to make the real-world process more efficient based on hash table structures?

- 11-B This one takes a little math. How many people need to be at a gathering to make it more likely than not that they share a birth month (not a birth *day*)? For this problem, assume that all twelve birth months are equally likely. If there is only one person at the gathering, then they must have a unique birth month. The second person will have a unique birth month with a likelihood of 11/12. The third person will have a unique birth month with a likelihood of 10/12, and so on. The likelihood that all the people have distinct birth months is the product of those likelihoods. Multiply them and find when the combined likelihood of having unique birth months becomes less than 50 percent.
- 11-C The idea of hashing can be used in sorting. In [Chapter 3](#), “[Simple Sorting](#),” and [Chapter 7](#), “[Advanced Sorting](#),” we introduced a number of sorting methods. Which ones use hashing or something like it to put the items in order?
- 11-D With the HashTableOpenAddressing Visualization tool, make a small quadratic hash table with a size that is *not* a prime number, say 24, and a maximum load factor of 0.9, so that it doesn’t grow. Fill it very full with, say 20, random items. Now search for nonexistent key values. Try different keys until you find one that causes the quadratic probe to go into an unending sequence. This repetitive sequence happens because the quadratic step size modulo the array size forms a repeating series.
- Repeat the experiment, but this time use a prime number for the array size, say 23. Can you find nonexistent keys that cause a similar unending sequence?
- 11-E With the HashTableChaining tool, create an array with 11 cells and a maximum load factor of 1.99 to allow high density. Next, fill it with 20 random items. Inspect the linked lists that are displayed. What is the longest list? Add the lengths of all these linked lists and divide by the number of lists to find the average list length. On average, you need to search this length in an unsuccessful search. (Actually, there’s a quicker way to find this average length. What is it?)

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 11.1 Implement a new method for `HashTable` that finds all the keys that were not placed at their initial hash position within an open addressing hash table due to collisions. Show the counts of displaced keys for the linear, quadratic, and double hash probes in hash tables with maximum load factors of 0.5, 0.7, and 0.9 (in other words, under nine different conditions: three probe schemes times three load factors).

The number of collisions depends heavily on the distribution of keys inserted in the table. You should run tests several times using randomly generated keys because the results will vary with each set. Make sure you use the same set of keys to insert in each of the different hash table types to make a reasonable comparison. You can generate 200 `random` integers in the range `[0, 999]` by importing the `random` module and evaluating

```
random.sample(range(1000), 200)
```

Using the `random.sample()` function guarantees that there will be no duplicate keys in the sequence. Initialize your hash tables with a size of 103 to lessen the likelihood of some probe sequences being unable to find empty cells in a small table. Run your tests many times to see whether some probe algorithms are clearly better or worse than others.

- 11.2 Write two hash functions that implement the digit-folding approach described in the “[Folding](#)” section of this chapter. One of the functions should fold groups of three digits and the other groups of two digits. Use these functions to create two `HashTable` objects with linear probing and the displaced key counting method from Project 11.1. Write a program that fills these two hash tables with a thousand random 10-digit integers generated by `random.sample(range(10000000000), 1000)`. Show the counts of displaced keys for the two hashing functions and the same three maximum load factors: 0.5, 0.7, and 0.9.

Accessing a group of `K` digits in a positive number may be easier than you think. Can you generalize the folding hash function to work with

any number of digits, or maybe even with any number for the folding range, not just 10^k ?

- 11.3 Explore what happens when the hash table size is a power of 2 instead of a prime number. Rewrite the `HashTable.__growTable()` method so that it doubles the size of the table without finding the next prime number larger than that. Use the same conditions as in Project 11.1, except use a starting size of 128 for the hash tables. The 200 keys that are inserted will force the table to grow at least once, and it should remain a power of 2.

Using a table size that is not a prime number increases the chances of collisions, so much so that you are likely to run into the exception raised by the `insert()` method when the probe sequence runs out of cells to try (see [Listing 11-5](#)). The exception will happen for only some distributions of keys, so you may need to seed the random number generator with different values to cause the exception. Make sure you catch the exception and record the problem for the particular probe sequence and load factor. The same set of keys may work with one probe sequence, but not in others.

As in Project 11.1, show the number of displaced keys for the nine different conditions: three probe schemes times three load factors. If the 200 keys cannot be inserted for a particular condition, show that too.

- 11.4 The double hashing step size calculation in [Listing 11-10](#) uses the `simplehash()` function. Replace that function with a multiplicative hashing function that is a variation on Horner's method described in the "[Hashing Strings](#)" section, except that it's designed to work with integer hash keys. The integer keys can be treated as a sequence of bytes. You can get the lowest byte from a big integer `N` by using a bit mask in Python, `N & 0xFF`. The loop iterates over the bytes and computes a hash that starts at 0. On each iteration, the current hash is multiplied by a prime, and the low byte plus another prime are added to get the next value of the hash. Multiplying by a prime and adding another prime help spread the influence of each bit of the key across the hashed value.

Produce a table like [Table 11-2](#) showing the insertion of 20 integer keys randomly selected from the range `[0, 99999]`. Show the multiplicative hashed address along with the modulo with the small prime to derive

the step size. You need to write some code to produce the probe sequence and peek at the stored values before inserting the item in the hash table in order to show the last column of the table.

- 11.5 Hash tables are perfectly suited to the task of counting things like the number of times words are used in a text. By going through the text a word at a time, you can check a hash table to see whether the word has already been seen before. If it hasn't, the word is inserted as a key in the hash table with a value of 1. If it has been seen, the table is updated to hold the incremented count. Traversing the completed hash table gets the overall word counts.

Write a program that reads a text file, extracts the individual words, counts the number of times they occur using a hash table, and then prints out a list of all the distinct words and their counts. To get the lines of a text file in Python, you can use a loop like `for line in open('myfile.text', 'r').` To get the words from the line, you can use a loop like `for word in line.split(),` which splits the string at whitespace characters. To trim off leading and trailing punctuation from a word, you can use the `strip()` method of strings, as in `word.strip('()<>[]{}-_.?!:;\"')`. This would convert "(open-addressing!)" to "open-addressing", for example. For case-insensitive word counting, you can use the `lower()` method for strings to make all the characters lowercase. Show the output of your program running on a short text file.

12. Spatial Data Structures

In This Chapter

- [Spatial Data](#)
- [Computing Distances Between Points](#)
- [Circles and Bounding Boxes](#)
- [Searching Spatial Data](#)
- [Lists of Points](#)
- [Grids](#)
- [Quadtrees](#)
- [Theoretical Performance and Optimizations](#)
- [Practical Considerations](#)
- [Further Extensions](#)

So far, all our data structures have stored records containing a key to identify them. That's perfect when you want to find a record that exactly matches a key. What about when you want to identify items by position—for example, finding all the grocery stores within a certain distance of a particular location? For that task, you need a structure that uses two separate numeric keys to identify the location of records that correspond to points in space.

Although this pair of numbers may seem to be a simple extension from one numeric key, the data structures must be dramatically changed to make the kinds of desired operations happen efficiently. The nature of points in space and their distances drives much of the design. We look first at the ways to represent points in space and then at three ways of storing records associated with those points. We wrap up with a discussion of the performance that you

can expect from these structures and some extensions needed for other domains.

Spatial Data

Data that has position attributes is called **spatial data**. Spatial data includes Cartesian coordinates and geographic coordinates.

Cartesian Coordinates

The simplest form of spatial data corresponds to points on a flat surface. Each point is represented by a pair of numbers that specify its X and Y coordinates on the surface, usually written as (x, y) . These (x, y) coordinates on a flat surface are known as **Cartesian coordinates**, invented by French mathematician René Descartes. For example, if you were programming a simulation of a football game, you could use Cartesian coordinates to represent the location of each player on the field. Because a football field is a very small piece of the surface of the Earth, you can conveniently pretend that the surface of the field is perfectly flat, so it is appropriate to use Cartesian coordinates.

Cartesian coordinates could be represented using any convenient unit of distance—for example, pixels on a screen, or feet or meters on a playing field. The exact units chosen are important to the application but do not affect the data structures and algorithms if the units are the same for both dimensions.

Geographic Coordinates

Spatial data points on the surface of the Earth are usually represented by a pair of numbers that specify the **latitude** (the angle north or south from the equator) and the **longitude** (the angle east or west from the prime meridian at Greenwich, England). Some applications also require a third value corresponding to the elevation on or above the surface of the Earth. In this chapter, we consider just points that are on the surface of the Earth, that is, at sea level.

[Figure 12-1](#) shows how the globe is divided into hemispheres by the equator in blue and the prime meridian in red. For example, the location of Ottawa, Canada, is approximately $(45.30096^\circ, -75.81689^\circ)$ as shown by the small green dot. The negative longitude indicates that the point is to the west of the

prime meridian. The location of Johannesburg, South Africa, is about $(-26.17643^\circ, 28.04551^\circ)$ marked by another dot. The negative latitude indicates that Johannesburg is south of the equator. Latitude, longitude coordinates are known as **geographic coordinates**.

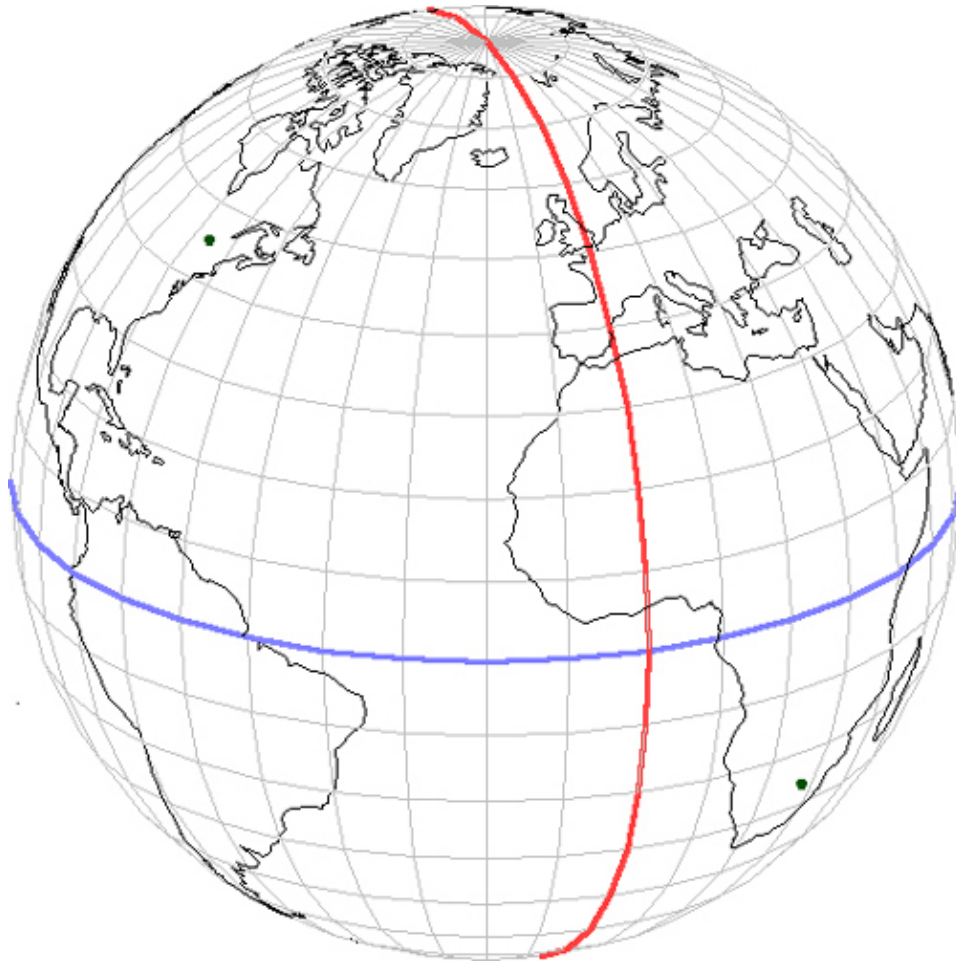


Figure 12-1 *Geographic coordinates use longitude and latitude*

Computing Distances Between Points

Later in this chapter, we look at data structures that support searching for records at or near specified Cartesian or geographic coordinates. To do these calculations, you need to be able to compute the distance between two spatial data points. This calculation is easy to implement for Cartesian coordinates and a little more complicated for geographic coordinates.

Distance Between Cartesian Coordinates

To compute the distance between two points specified by Cartesian coordinates, you make use of the familiar Pythagorean theorem that the length of the hypotenuse squared is the sum of the squares of the sides of a right triangle. Because the points are defined by their coordinates in the X and Y axes, you can align a right triangle with the axes to determine what is sometimes called the **Euclidean distance**. If the two points are at (x_1, y_1) and (x_2, y_2) , then the Euclidean distance d between the two points is

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Implementing this distance measure is simple in Python, as shown in [Listing 12-1](#). The result of this formula is in the same units of measurement as the input Cartesian coordinates.

Listing 12-1 *Euclidean Distance Between Cartesian Coordinates*

```
# return the Euclidean distance between points (x1,y1) and (x2,y2)
def euclideanDistance(x1, y1, x2, y2):
    dx = x2 - x1
    dy = y2 - y1
    return (dx*dx + dy*dy) ** 0.5
```

Distance Between Geographic Coordinates

Calculating the distance between geographic coordinates is more complex. First, geographic coordinates are specified as angles of latitude and longitude on the Earth—which do not directly correspond to distances expressed in units of length. Second, the surface of the Earth is neither flat nor perfectly spherical, which further complicates the computation. There are many ways that distance between geographic coordinates can be calculated. The accuracy of the calculations depends on the simplifying assumptions and approximations used.

The shortest path between two points on a sphere lies along the curve of a **great circle**. To get the great circle, take the plane that contains the two points and the center of the sphere and intersect it with the sphere itself. Computing the distance between Ottawa and Johannesburg using the Euclidean distance would be useful only if you could travel in a straight line through the Earth (see [Figure 12-1](#)). Even a path on the surface of the Earth that seems to go directly

between the two points is not necessarily the shortest path. To be sure that the path is the shortest, it must lie on the great circle path that connects them. That is why a flight from New York to Spain flies in a path that curves toward the North Pole, rather than flying due east, as a typical map might imply.

The **haversine formula** computes the great circle distance between two geographic coordinates on a sphere. Although Earth is not a perfect sphere, the haversine formula will give correct answers to within 0.5 percent. That should be adequate for an application such as finding the nearest airport or coffee shop, but possibly not accurate enough for others, such as determining exactly how many kilometers of cable would be needed to connect two airports.

The haversine formula for the distance between two geographic coordinates is

$$d = 2 R \arcsin \sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos \phi_1 \cos \phi_2 \sin^2 \left(\frac{\Delta \lambda}{2} \right)}$$

where

- R is the radius of the Earth
- ϕ_1 and ϕ_2 are the latitudes of the two points
- $\Delta \phi$ is the difference between the latitudes of the two points
- $\Delta \lambda$ is the difference between the longitudes of the two points.

Although this formula looks rather daunting, the Python implementation is straightforward, as shown in [Listing 12-2](#).

Listing 12-2 *Python Implementation of the Haversine Distance*

```
from math import *
RADIUS_OF_EARTH = 6371      # radius in kilometers

# return the haversine distance between (lon1, lat1) and (lon2, lat2)
def haversineDistance(lon1, lat1, lon2, lat2):
    lat1 = radians(lat1)      # trig functions need radians
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)
```

```
dLon = lon2 - lon1      # difference of longitudes
dLat = lat2 - lat1      # difference of latitudes

# Haversine formula:

a = sin(dLat/2)**2
if dLon != 0: # save some trig for dLon == 0
    a += cos(lat1) * cos(lat2) * sin(dLon/2)**2

# Numerical issues at antipodal points requires min, max:
return 2 * RADIUS_OF_EARTH * asin(min(1, max(a, 0)**0.5))
```

It is worth noting that as commonly spoken in English, geographic coordinates are referred to as “latitude, longitude” pairs. When you’re writing code to uniformly support either Cartesian or geographic coordinates, however, it makes more sense to refer to geographic coordinates as “longitude, latitude” pairs. Simply put, longitude angles correspond to values along the equator, analogous to points along the Cartesian X axis. Likewise, latitude angles correspond to values along the north-south meridians, analogous to the Cartesian Y axis. The methods in the class definitions we present for spatial data structures accept “*a*, *b*” pairs to specify spatial location, which are then interpreted appropriately to mean either *x*, *y* or longitude, latitude coordinates. Although the Euclidean distance will work with any units, this definition of the haversine distance always returns the distance in kilometers because the Earth’s radius is specified that way.

Circles and Bounding Boxes

When working with spatial data, you frequently need to determine whether points lie within a particular area. For example, you might want to know whether a particular geographic coordinate lies within a particular country or within a property boundary. Many kinds of regions might be checked. The two simplest and most common ones are circular and box-shaped areas. You will need to use the distance calculations in many cases to determine whether a point is inside or outside one of these regions.

Clarifying Distances and Circles

When you begin the search for a data point nearest to a query point, and no data point has been considered yet, it is convenient to think of the initial distance

from the query point to the nearest point as being at infinity—in other words, a number that is arbitrarily large. The Python `math` module conveniently provides a predefined constant `math.inf`, which is larger than any other int or float number. (In versions of Python before 3.5, the constant for infinity was produced by the expression `float('inf')`, which can still be used.) As the search proceeds, and subsequent points are encountered, that distance becomes a finite number, which gets successively smaller as points closer to the query point are identified.

In Cartesian coordinates, at any step of the search, you can think of the query point together with the distance to the nearest point so far as defining a circle around the query point, whose radius is equal to the distance to the current nearest point.

In geographic coordinates, a curve of constant distance is not a perfect circle when drawn on a grid whose latitude lines and longitude lines are spaced at uniform degrees of separation (known as the Mercator projection). Indeed, this is what causes the apparent distortion of landmasses when viewed on some maps. For example, near the poles, the meridians are much closer together than they are near the equator.

These changes in distance are illustrated in [Figure 12-2](#) where three 1,000 km paths have been drawn in blue. The same paths are drawn on the globe as it appears from space and on a map using the Mercator projection with 10 degrees separating each of the longitude and latitude grid lines. The path around Iceland looks much larger than the path centered on the coast of Brazil in the Mercator projection, even though both are equal diameter. The path surrounding the tip of Tierra del Fuego, the southern tip of South America (only partially visible on the globe projection), is also the same diameter.

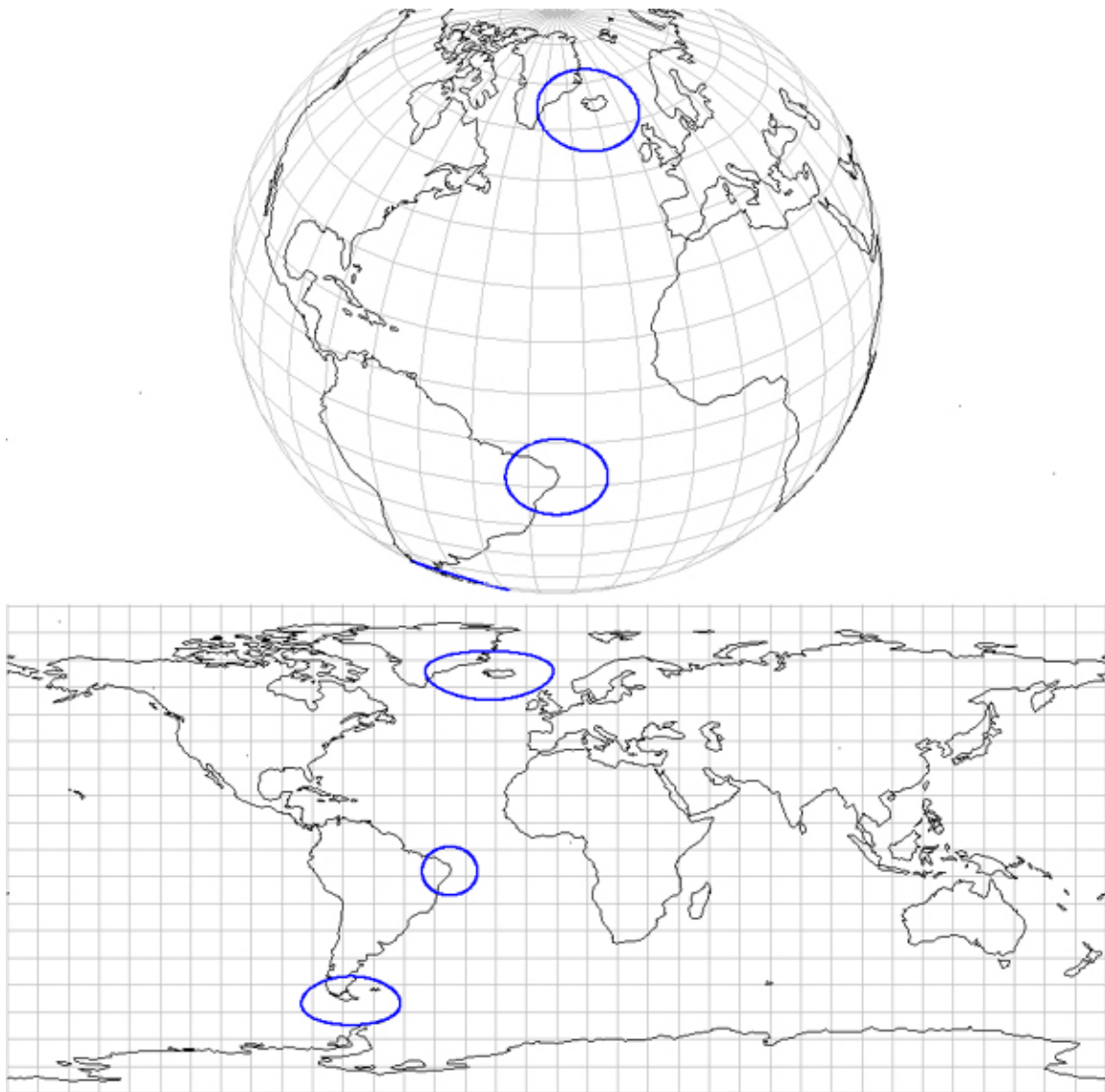


Figure 12-2 *Comparing 1000 km circles at different points on the globe*

In the following pages when we discuss a *query circle*, keep in mind that sometimes the circle is not truly circular. The term *query circle* is used to refer to a curve of constant distance from a query point, which may not always look perfectly circular depending on the map viewpoint. Our computations account for this apparent distortion.

Bounding Boxes

As we just mentioned, when using geographic coordinates, a query circle with a radius of constant distance does not look like a circle when projected on a

map grid with uniform latitude and longitude spacing.

For geographic coordinate query circles far from the equator, the circle begins to look more like an ellipsoid. In addition, the widest part of the query circle is not directly due-east or due-west of the center of the query circle as it is at the equator or in Cartesian coordinates. For example, as a search circle gets farther north from the equator, the circle gets increasingly flattened, and the widest part of the search circle progresses farther north of the center of the circle.

In the coming pages we examine data structures that partition space into rectangular pieces. To search for a particular point, the algorithms repeatedly check whether a query circle intersects a particular rectangular piece of space containing data points. It can be computationally complex to determine whether a circle and rectangle intersect, and, even more so, whether a distorted ellipsoid and a rectangle intersect.

One way to simplify these repeated query circle-rectangle intersection calculations is to approximate the query circle by a rectangular bounding box that snugly encloses the query circle in either Cartesian or geographic coordinates. As you will see, determining whether two rectangles intersect is a fast and simple computation. If the bounding box of the query circle doesn't intersect a rectangle of space, then you can be certain that there's no chance that the query circle itself intersects the rectangle of space. Only when the bounding box does intersect a rectangle of space do you need to do more detailed calculations to attempt to find a point within that rectangle that matches or is nearby the query point.

The Bounding Box of a Query Circle in Cartesian Coordinates

In Cartesian coordinates, you can easily identify the left, right, top, and bottom coordinates of the bounds of a query circle of radius r , with a center at a, b , as shown in [Figure 12-3](#). The left boundary lies at $a - r$. The right boundary lies at $a + r$. The vertical boundaries are similar.

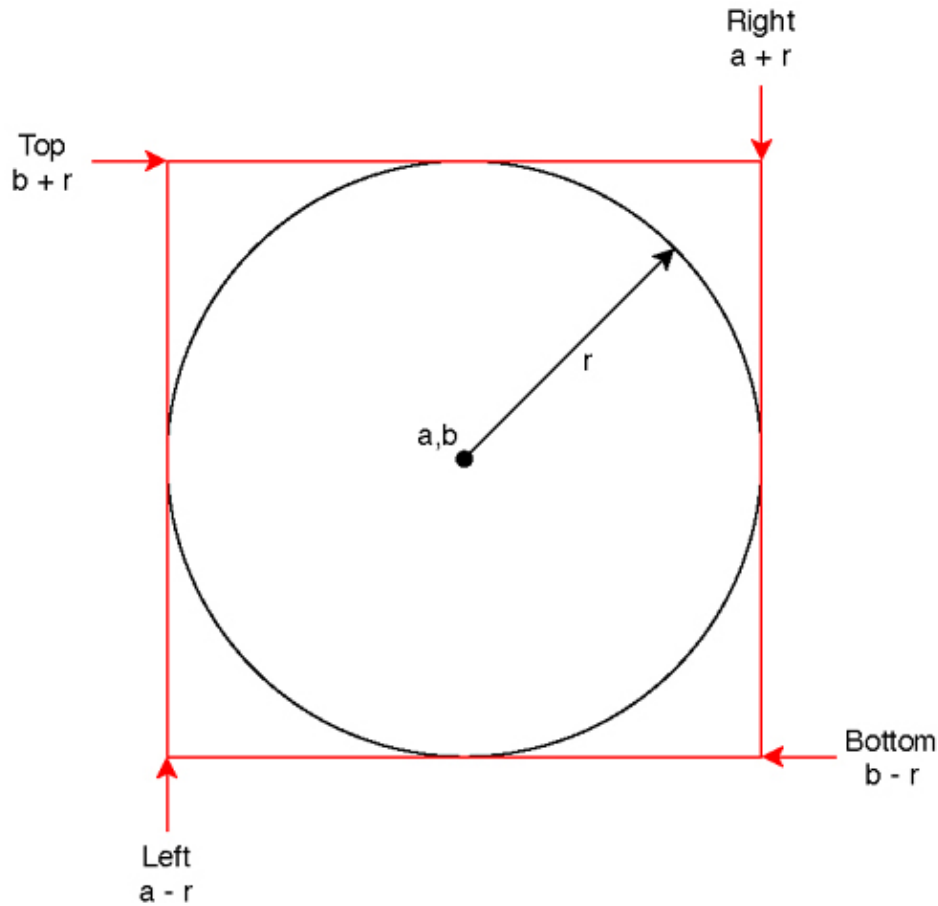


Figure 12-3 *The bounding box of a circle in Cartesian coordinates*

The Bounding Box of a Query Circle in Geographic Coordinates

How do you determine the bounding box of a query circle in geographic coordinates? It's clear that you need to calculate the bounds for longitude and latitude, but how? The formulas that solve this problem (shown here) employ methods in spherical geometry that are outside the scope of this text.

$$\Delta_{long} = \arcsin\left(\frac{\sin\left(\frac{r}{R}\right)}{\sin\left(\frac{\pi}{2} - B\right)}\right)$$

$$\Delta_{lat} = \frac{r}{R}$$

You do not need to understand the detailed math to create the methods that compute and use the bounding box of a search circle in geographic coordinates. The preceding equations and the diagram in [Figure 12-4](#) show how to compute the width in longitude and height in latitude from the center of the query circle to the edges of its bounding box, where

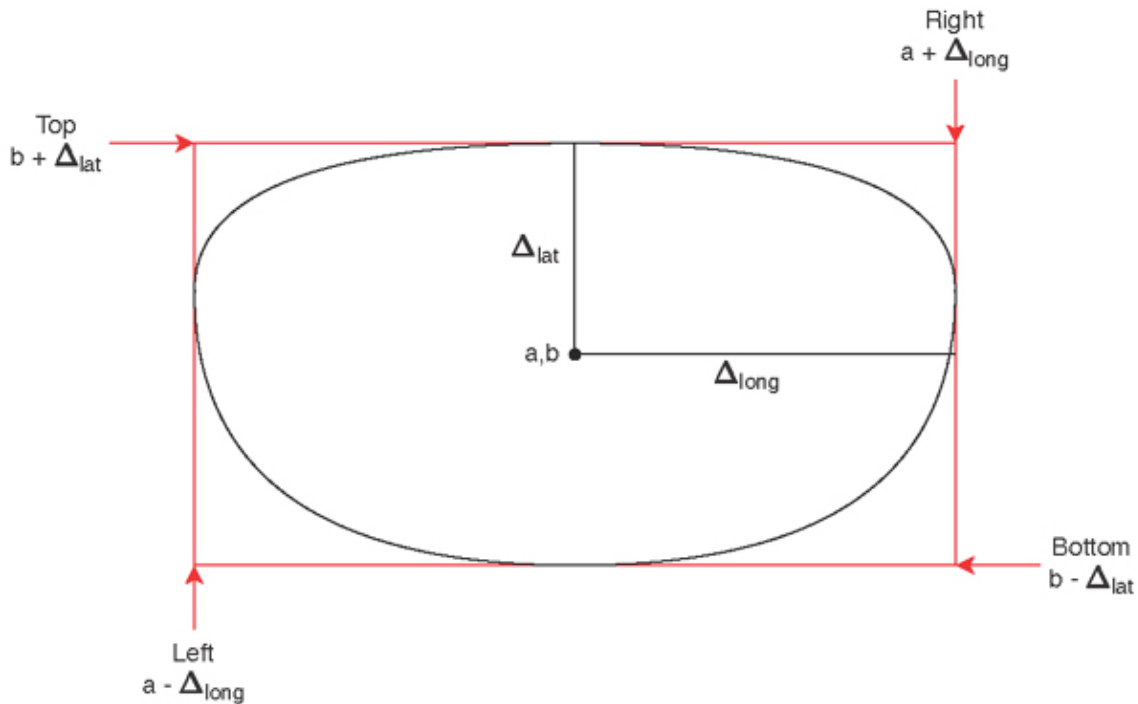


Figure 12-4 *The bounding box of a query circle in geographic coordinates*

1. r is the radius of the query circle.
2. R is the radius of the Earth.
3. B is the latitude b of the a, b center of the query circle, expressed in radians instead of degrees.
4. (Δ_{long}) is the longitude width in radians from the center to the widest point of the query circle.
5. (Δ_{lat}) is the latitude height in radians from the center to the top or bottom of the query circle.

Note that the widest point of the query circle may not be due east of the center. The example shown in the figure illustrates this distortion resulting in the widest part of the circle being slightly vertically offset from the latitude, b .

Implementing Bounding Boxes in Python

We represent bounding boxes of all types in Python using the `Bounds` class and its `CircleBounds` subclass, as shown in [Listing 12-3](#).

Listing 12-3 *Creating and Adjusting Bounds Objects*

```
class Bounds(object):
    def __init__(self, left = -inf, right = inf,
                  top = inf, bottom = -inf):
        Bounds.adjust(self, left, right, top, bottom)

# mutator to initialize/change bounds in existing object
def adjust(self, left, right, top, bottom):
    self.__l = left
    self.__r = right
    self.__t = top
    self.__b = bottom

# return a new Bounds, where some of the
# current boundaries have been adjusted
def adjusted(self, left = None, right = None,
             top = None, bottom = None):

    if left == None: left = self.__l
    if right == None: right = self.__r
    if top == None: top = self.__t
    if bottom == None: bottom = self.__b

    return Bounds(left, right, top, bottom)
```

Instances of the `Bounds` class shown in [Listing 12-3](#) store the coordinates of the left, right, top, and bottom boundaries of a rectangular section of Cartesian or geographic space. You can create a new `Bounds` object by invoking the constructor as shown in this snippet:

```
b = Bounds(-125.6, -60.8, 49.72, 23.77)
```

The returned object represents the rough rectangular boundaries of the continental United States, with its left boundary at -125.6 degrees longitude, its right boundary at -60.8 degrees longitude, its top boundary at 49.72 degrees latitude, and its bottom boundary at 23.77 degrees latitude.

Invoking `Bounds()` with no arguments results in a rectangle with infinite extent. The mutator method, `adjust()`, enables the client code and the constructor to update the private attributes.

A separate method called `adjusted()` creates and returns a *new* `Bounds` object derived from the current object, and with some or all the boundaries replaced with new values.

The CircleBounds Subclass

The `CircleBounds` subclass of `Bounds`, shown in [Listing 12-4](#), creates a box enclosing a circle.

Listing 12-4 *Initializing and Adjusting CircleBounds objects*

```
# A bounding box that surrounds a search circle. Properly handles
# distorted circles resulting from geographic coordinates.
class CircleBounds(Bounds):
    # create the bounding box that surrounds the search
    # circle at a,b,r using the specified distance function
    def __init__(self, a, b, r, func):
        # remember the circle's parameters for later
        self.__a, self.__b, self.__r = a, b, r
        self.__func = func

        # get the width/height of the bounding box
        deltaA, deltaB = self.__deltas(a, b, r, func)

        # initialize the superclass's rectangular bounds
        super().__init__(a - deltaA, # left
                        a + deltaA, # right
                        b + deltaB, # top
                        b - deltaB) # bottom

# if the circle's radius changed, mutate its bounds
def adjust(self, r):
    if r != self.__r:
        self.__r = r

        # new dimensions of bounding box
        deltaA, deltaB = self.__deltas(
            self.__a, self.__b, r, self.__func)

        # update the box
```

```
super().adjust(self.__a - deltaA, # left
               self.__a + deltaA, # right
               self.__b + deltaB, # top
               self.__b - deltaB) # bottom
```

The user creates a new `CircleBounds` object by invoking the constructor with the a , b coordinates of a circle center, the circle's radius, and a reference to a distance function, as shown in this snippet:

```
c = CircleBounds(38.736, -9.142, 3.14, haversineDistance)
```

This snippet creates a `CircleBounds` object that represents the rectangular bounds surrounding a circle centered in Lisbon, Portugal, with a radius of 3.14 km.

As shown in [Listing 12-4](#) the constructor saves the parameters in private attributes—to potentially be used later by the `adjust()` mutator method that adjusts the radius of the circle. Given the a , b coordinates and radius r of the circle, the constructor obtains the distances from the center of the circle to the widest and tallest points on the circle, called `deltaA` and `deltaB`. Finally, the constructor uses those two distances to invoke the superclass's constructor with the rectangular bounds of the circle.

The `adjust()` mutator can be invoked by the client with a new value for the circle's radius. If `adjust()` determines that the radius has changed, then `deltaA` and `deltaB` are recomputed, and then the superclass's `adjust()` method replaces the rectangular bounds with the newly computed boundaries.

All that remains to discuss is how the `deltaA` and `deltaB` distances are computed, as shown in the `__deltas()` method in [Listing 12-5](#). The method implements the calculations for width and height of the two types of circles shown in [Figure 12-3](#) and [Figure 12-4](#). For infinite radius circles or Cartesian coordinates, `deltaA` and `deltaB` are simply the radius of the search circle. For geographic coordinates, `deltaA` and `deltaB` are computed using the preceding complex formulas and substituted for Δ_{long} and Δ_{lat} in [Figure 12-4](#), respectively.

Listing 12-5 *Computing the Width and Height of a Bounding Box Around a Circle*

```
class CircleBounds(Bounds):
...
```

```

def __deltas(self, a, b, r, func):
    # When r is infinity, bounds are always infinite
    # For cartesian coordinates, the edges of the bounding
    # box are r away from the center of the search circle.
    if r == inf or func == euclideanDistance:
        deltaA = deltaB = r

    else: # geographic coordinates
        # The width of the bounding box is determined by
        # the width of the distorted circle at its widest point.
        alpha = r / RADIUS_OF_EARTH
        gamma = radians(90 - abs(b)) # b is latitude

        # if the circle itself crosses the pole then
        # alpha >= gamma, so we set deltaA as inf,
        # effectively treating the width as if it were inf
        if alpha >= gamma:
            deltaA = inf
        else:
            # the circle doesn't cross a pole, so get
            # longitude angle from center of circle at
            # the circle's widest width calculated using
            # a spherical right triangle identity.
            deltaA = degrees(asin(sin(alpha) / sin(gamma)))

        # latitude angle directly above/below the
        # center of the circle. This works since above
        # or below is directly on a meridian.
        deltaB = degrees(alpha)

    return deltaA, deltaB

```

Recall that for geographic coordinates, a query circle gets progressively flattened as it approaches either pole. If the center of the circle gets within the query radius of a pole, the circle extends beyond the pole (in other words, the pole lies within the query circle). This condition is detected in the code when the `alpha` variable exceeds the `gamma` variable, which could result in an undefined value of the inverse sine function used to compute `deltaA`. In that case, instead of computing the inverse sine, it sets `deltaA` to be infinite because the search circle wraps all the way around the pole.

Determining Whether Two Bounds Objects Intersect

Later in this chapter in the discussion of the `Grid` and `QuadTree` classes, some methods need to determine whether two rectangles defined by their `Bounds` objects intersect. You can determine whether two rectangles intersect by making the simple observation that they do *not* intersect if one rectangle is fully to the left of the other or fully above the other.

As illustrated in [Figure 12-5](#), rectangles 2, 3, and 4 do not intersect with rectangle 1 because they are each either directly above or to the side of rectangle 1. Rectangle 5, however, does intersect with rectangle 1.

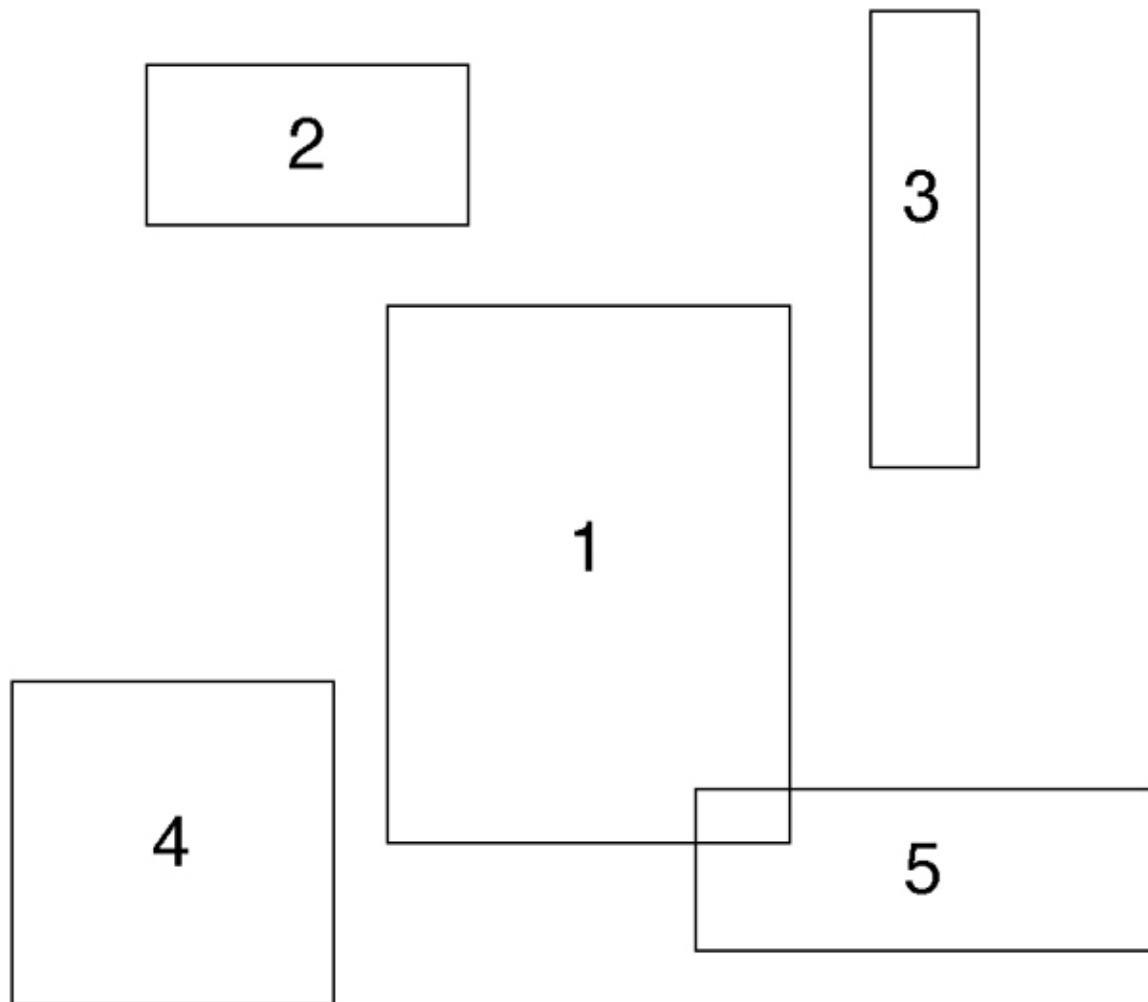


Figure 12-5 *Intersecting and nonintersecting rectangles*

For any two rectangles A and B, you can say that they *don't* intersect if the following pseudocode expression is true:

(A is to the left of B) **or** (B is to the left of A) **or**

(A is above B) **or** (B is above A)

So, to determine whether rectangles A and B *do* intersect, you simply negate that entire expression and simplify, resulting in

(A is NOT to the left of B) **and** (B is NOT to the left of A) **and**

(A is NOT above B) **and** (B is NOT above A)

Examining rectangles 1 and 5, you can see that this latter expression evaluates to True, and therefore that rectangles 1 and 5 *do* intersect.

The code in [Listing 12-6](#) shows the simple implementation of the `intersects()` method. It uses the negation of the definition of *not* intersecting.

Listing 12-6 *Determining if Bounds Objects Intersect*

```
class Bounds(object):
...
    # return True if bounds of the rectangles intersect.
    # Two rectangles don't intersect if one is totally to
    # the left of or above the other. So we negate to
    # determine if they do intersect.
    def intersects(self, other):
        return not(
            # check if self's box is to left or right of other
            self.__r < other.__l or self.__l > other.__r or

            # check if self's box is above or below other
            self.__b > other.__t or self.__t < other.__b)
```

Determining Whether One Bounds Object Lies Entirely Within Another

As you will soon see, some methods also need to determine whether the bounds of a circle fit entirely within the bounds of a square subset of cells on a grid.

[Figure 12-6](#) illustrates a sample scenario, where you want to determine whether the circle drawn in black, with its bounding box drawn in red, lies entirely within the blue-shaded square of grid cells. If you have a `Bounds` object initialized with the boundaries of the blue shaded area, and another `CircleBounds` object initialized with the bounding box surrounding the circle,

it is simple to determine whether the circle fits within the blue box, as shown in the `within()` method in [Listing 12-7](#).

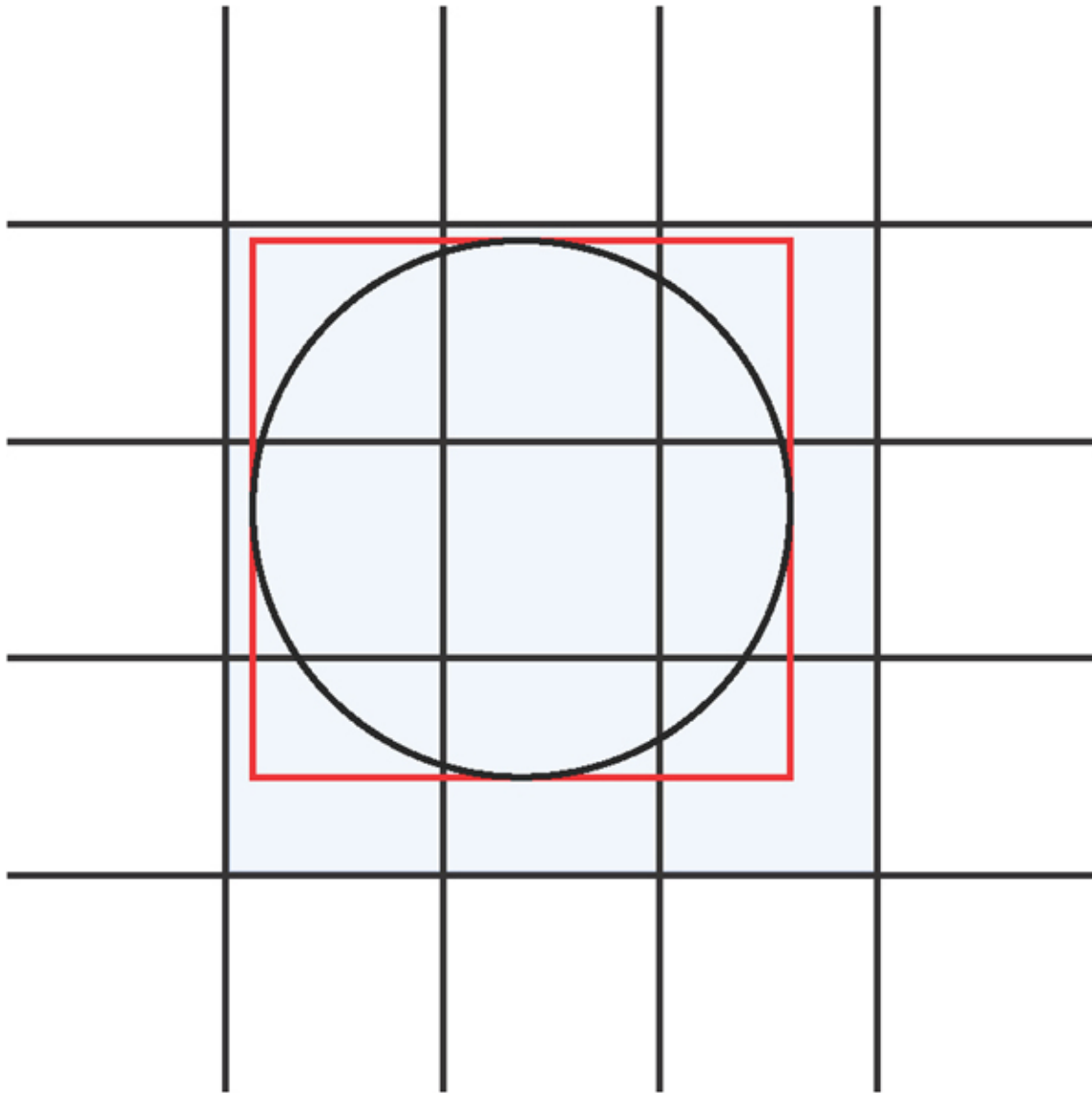


Figure 12-6 *A CircleBounds within another Bounds object*

Listing 12-7 *Determining Whether a Bounds Object Lies Within Another Bounds Object*

```
class Bounds(object):  
...  
    # return True if the bounds of self fall within other  
    def within(self, other):  
        return (self.__r < other.__r and
```

```
self.__l > other.__l and  
self.__t < other.__t and  
self.__b > other.__b)
```

Note that the same definition of `within()` is used for both `Bounds` and `CircleBounds` objects. This is good enough for the purpose of searching the data structures but would not correctly determine whether the *circle* of one `CircleBounds` object was truly within (or even overlapping) the *circle* of another `CircleBounds` object.

Searching Spatial Data

Now that we've handled these geometry basics, we can focus on the interesting challenge of searching a collection of spatial data points to find an exact match to a query point or to find the closest match to a query point. To find the closest match, we need to specify what it means to be close; in other words, which distance function should be used to compare a query point to a candidate data point? If the spatial data consists of Cartesian coordinates, the `euclideanDistance()` function will be appropriate. If the spatial data consists of geographic coordinates, the `haversineDistance()` function will be appropriate.

In the following sections, we look at several classes that implement data structures and algorithms to support exact match and closest match spatial search. The classes vary in code complexity and Big O performance, so their applicability depends on the amount and distribution of data you are attempting to process. The important issues are how to deal with using coordinates as keys and how to deal with distance when searching. First, we start with a simple approach. Then we add more complex structures to improve performance. Finally, we look at the trade-offs between the different approaches.

Lists of Points

If you want to support spatial data search of a small number of points, it can be appropriate to just store the spatial data points in one of the structures you've already seen, such as an array or a list. Then the spatial data is queried by traversing over the items to find either an exact match or the closest match to the query point. This brute-force approach is the simplest possible solution, though because of that simplicity, the Big O performance is $O(N)$ to search for

an exact or nearest match. In other words, to do a spatial search using brute force, the code must compare the query point to every point stored in the structure.

Creating an Instance of the PointList Class

We'll use a simple Python list (array) to hold a collection of items in a `PointList` class. Client code creates an instance of the `PointList` class by invoking the constructor to create an object that contains Cartesian points:

```
b = PointList('cartesian')
```

or that contains geographic points:

```
b = PointList('geographic')
```

If the constructor is passed any string other than `'cartesian'` or `'geographic'`, an exception will be raised.

[Listing 12-8](#) shows the code for the `PointList` constructor. There are just two private attributes: a reference to the distance function appropriate to either Cartesian or geographic coordinates, and a reference to an initially empty list that will be used to store the points in the object.

Listing 12-8 *The PointList Constructor*

```
class PointList(object):
    def __init__(self, kind): # must specify type of coordinate
        if kind == 'cartesian':
            self.__distance = euclideanDistance
        elif kind == 'geographic':
            self.__distance = haversineDistance
        else:
            raise Exception("Invalid kind of coordinate")

        self.__points = [ ] # keep track of inserted points
```

Inserting Points

Points are inserted into the `PointList` object using the `insert()` method. For Cartesian points, the user passes to the `insert()` method an *x* coordinate, a *y*

coordinate, and a reference to whatever data will be associated with the Cartesian x, y point:

```
b.insert(34, 56, "Player 1") # insert a point at x == 34, y == 56
```

Likewise, for geographic points, the user passes to `insert` a longitude, a latitude, and a reference to the data associated with this geographic longitude, latitude point:

```
b.insert(-73.987853, 40.748440, "Empire State Building")
```

Most likely, the client code will use a loop to repeatedly invoke `insert()` so that the `PointList` object contains all the points to be searched.

The implementation of the `PointList.insert()` method is quite simple (see [Listing 12-9](#)). Each inserted point is stored as a tuple containing three elements appended to the end of the private `__points` list. Each tuple contains

- The x , or longitude, coordinate
- The y , or latitude, coordinate
- A reference to the data to be associated with the specified point

Listing 12-9 *Inserting a Point into a `PointList`*

```
class PointList(object):  
...  
  
    def insert(self, a, b, data):  
        # Loop through the points looking for an exact match  
        for i in range(len(self.__points)):  
            p = self.__points[i]  
            if p[0] == a and p[1] == b:          # Replace data  
                self.__points[i] = (a, b, data) # for a duplicate  
                return  
  
        self.__points.append((a, b, data))    # not there, so append
```

For all the data structures in this chapter, the `insert()` method ensures that there are no points stored with duplicate a, b coordinates. Thus, for `PointList`, `insert()` loops through all the points in the list, and if it finds an already-existing point with the same a, b coordinates, the data currently stored for that point is replaced with the newly inserted data. If no duplicate is found, then the

insertion is accomplished by appending a point to the end of the `__points` list. When the client later attempts to find an exact or nearest match, the `findExact()` or `findNearest()` methods will return the single point that is found to match the query point, and the returned data will be the data that had been most recently inserted for those coordinates. Because `insert()` loops over all the points in the list to avoid inserting a point with duplicate coordinates, it takes $O(N)$ time for each insertion.

Finding an Exact Match

You call the `findExact()` method to search for the point contained in the `PointList` object that exactly matches the specified x, y or longitude, latitude coordinates. If an exact match is found, the method returns a reference to the data that was stored for the matching point:

```
>>> ans = b.findExact(-73.987853, 40.748440)
>>> print("Answer was:", ans) # Answer was: Empire State Building
```

Not surprisingly, the implementation of `findExact()` shown in [Listing 12-10](#) is simple. The method loops through potentially every point stored in the `__points` list. If an exact match to the specified a, b coordinates is found, then the data for that point is returned to the client.

Listing 12-10 *Finding an Exact Match in a PointList*

```
class PointList(object):
...
    def findExact(self, a, b):
        for p in self.__points:
            if p[0] == a and p[1] == b:
                return p[2]
        return None
```

Because `findExact` must potentially loop through every point of the `__points` list in the worst case, the complexity of the method must be $O(N)$.

Deleting a Point

It is straightforward to implement a method to delete from the `PointList` a point specified by a pair of coordinates. As shown in [Listing 12-11](#), the approach is very similar to that used by `findExact()`, except that after the matching point is identified, it is removed from the list using Python's `del` operator. If a matching point is found, then `delete()` returns the data associated with the deleted point. Otherwise, `delete()` returns `None`.

Listing 12-11 *Deleting a Point from a `PointList`*

```
class PointList(object):
...
    # delete the point at a,b
    # Return the deleted point's data if found, or None
    def delete(self, a, b):
        # Loop through the points looking for an exact match
        for i in range(len(self.__points)):
            p = self.__points[i]
            if p[0] == a and p[1] == b: # found a match
                del self.__points[i]    # delete the point
                return p[2]             # return its data

        return None                    # point wasn't there
```

Just like `findExact()`, the Big O time for `delete()` is $O(N)$.

Traversing the Points

Because `PointList` is implemented as a simple list of tuples, traversing the points is trivial. As with the other data structures you've seen, `traverse()` is implemented as a generator as shown in [Listing 12-12](#).

Listing 12-12 *The `traverse()` method of `PointList`*

```
class PointList(object):
...

    def traverse(self):
        for p in self.__points:           # yield each of the tuples
            yield p
```

The implementation loops through each of the elements of the list, yielding the tuples containing the a , b coordinates and the associated data. Because every point in the list is visited once, the complexity of `traverse` is $O(N)$. The caller can use tuple-assignment in its `for` loop to assign the values to three variables, or just assign the returned tuple to a single loop variable.

Finding the Nearest Match

You call the `findNearest()` method to find the point stored in the `__points` list that lies closest to the query point. Depending on whether the `PointList` object stores Cartesian or geographic coordinates, `findNearest()` automatically uses the appropriate function to compute the distance from the query point to each of the object's points, as shown in [Listing 12-13](#).

Listing 12-13 *Finding the Nearest Point in a `PointList`*

```
class PointList(object):
...
    def findNearest(self, a, b):          # find closest point to (a,b)
        if len(self.__points) == 0:      # No points yet?
            return None

        distance = inf                   # Assume no nearest point
        for p in self.__points:          # For each point
            newDist = self.__distance(a, b, p[0], p[1])
            if newDist == 0:              # Nothing could be closer!
                return p[0], p[1], p[2], 0
            if newDist < distance:        # If it is closest so far,
                distance = newDist        # remember the distance,
                answer = p                # and the point

        return answer[0], answer[1], answer[2], distance
```

The method returns to the client a four-element tuple containing the x /longitude, y /latitude, the associated data, as well as the distance to the point that is closest to the query point. The distance function that was set when the object was constructed determines the units of the measurement.

For example, assuming that a geographic point for the location of the Eiffel Tower has already been inserted in the `PointList` object, this snippet shows what would be returned by an invocation of `findNearest()`:

```
>>> ans = b.findNearest(2.2, 48.8)
>>> print(ans)    # (2.294694, 48.858093, 'Eiffel Tower', 9.47495437535)
```

The implementation of `findNearest()` shown in [Listing 12-13](#) starts by checking the base case of an empty list. If the `PointList` is empty, there is no nearest point, so `None` is returned. If there are some points, it initializes the `distance` variable to infinity and then loops through all the points. If the distance from a, b to one of the new points is less than the current best distance, it updates the `answer` and then records the new best distance.

This approach is not efficient because it is necessary to compute the distance between the query coordinates and every point stored in the `PointList` object's `__points` list. The only shortcut happens when the query point has the exact same coordinates as one of the points in the `__points` list, in which case there would be no reason to keep looping through the remaining points and it returns the current point as the answer. The complexity of `findNearest()` is also $O(N)$ for each invocation.

Grids

The `PointList` class is appropriate to store and query only a relatively small number of spatial data points. If you have many points and they are mostly uniformly spread out in space, then you can superimpose a grid on top of the points to speed up the `insert()`, `findExact()`, and `findNearest()` methods. To see why this is so, consider [Figure 12-7](#), which shows a collection of 2,000 spatial data points, with a query point shown as a red diamond.

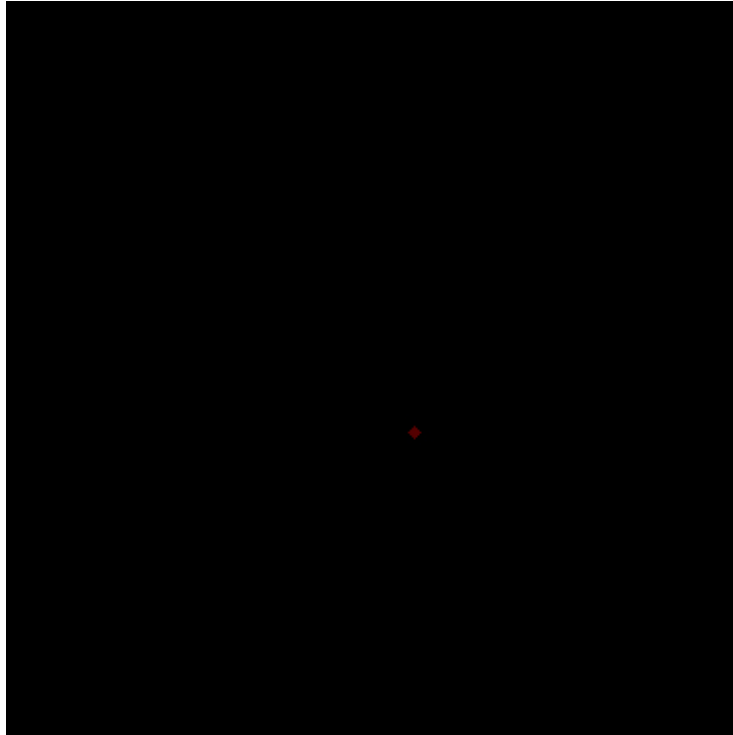


Figure 12-7 *Two thousand data points and a query point*

To find a point that exactly matches or is closest to the red query point, every one of the 2,000 points must be compared to the red query point. If you superimpose a grid on top of the points, you can break them up into 10 rows and 10 columns of cells, as in [Figure 12-8](#).

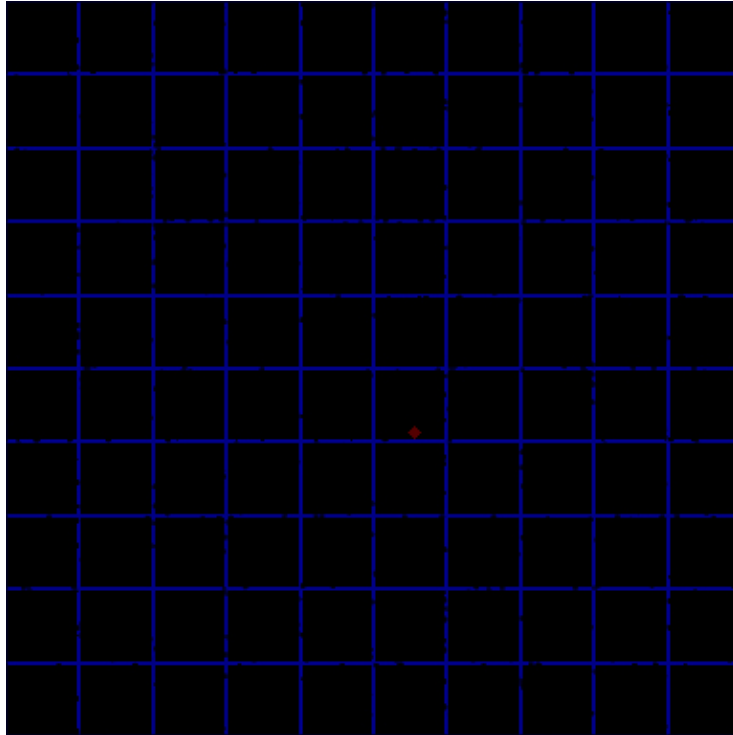


Figure 12-8 *Breaking up space into a 10×10 grid*

To find whether there's an *exact* match to the red query point, you only need to check the points that fall within the same cell as the query point. Finding the *nearest* point to the red query point is more complicated because you might end up checking the points in some neighboring cells. Even in this case, it is only necessary to check a small fraction of the total number of points.

Implementing a Grid in Python

Transforming Cartesian or geographic coordinates to the row and column numbers of a grid only requires that the coordinates are divided by the cell size (which is specified in units appropriate to the type of coordinates). The grid cells can be represented as an array to quickly access the list of points associated with each cell.

A two-dimensional array seems like the natural choice to hold the separate point lists. The two coordinates can be computed separately and used to index a specific cell in the array. Some issues arise, however, in dealing with negative values and the range of possible coordinates. Most programming languages do not allow negative indices in arrays, so if you converted -23.75 to, say, -24 , you couldn't use that to index a row or column.

To work around that limitation, you need to know the full range of possible coordinates that might be used. Knowing the bounds of the coordinates, you could subtract the minimum value of each coordinate from any given coordinate being processed to get a zero or positive value. The positive value can then be converted to an index in the array.

Knowing the coordinate bounds solves one problem but presents another. The bounds can be much larger than what's needed for a particular set of points. In geographic coordinates, the bounds are well known. Longitude ranges from -180 to $+180$, and latitude from -90 to $+90$. Let's assume that you want to track the position of every known ship on Earth in a data structure. If you organize them in a two-dimensional array, they will only be placed in some of the cells. Every cell that contains only land will be empty (except for maybe a ship being transported over land on a trailer). Even among the cells representing oceans and seas, there could be many cells without a single ship. That means a two-dimensional array could have a significant fraction of unused cells.

As you saw in [Chapter 11](#), hash tables can deal with both issues. The key provided to a hash table can include negative numbers or even a sequence of numbers like the a, b coordinates of a point. The hashing function transforms the coordinate pair into a large number and then maps the number to one of the cells in its table. As more items are stored in the hash table, the memory used to store it can grow while limiting the unused cells to a certain percentage of the total cells. Because the bounds on the coordinates are unimportant, a hash table can represent an infinite grid with positive and negative row and column numbers. This capability makes it well suited to managing the points in the grid data structure.

The next data structure for storing point data, `Grid`, uses a Python dictionary (hash table) whose key is a tuple containing the grid row number and column number (not the coordinates directly). The data stored in each grid cell is a list of points, just like the `list` used in `PointList`. When a dictionary is used to implement the storage of the grid, unused cells are eliminated, negative grid coordinates are handled well, and there is no need to determine the extreme bounds of the coordinates.

Creating an Instance of the Grid Class

You can create an instance of the `Grid` class by invoking the appropriate constructor to create an object that contains either Cartesian points or geographic points. The invocation must include an extra parameter, which is the width and height of each cell. For example, the following snippet creates a `Grid` instance that will contain points with Cartesian coordinates and with grid cells that are 10 units wide and high:

```
g = Grid('cartesian', 10)
```

Likewise, the following code creates a `Grid` instance that will contain points with geographic coordinates, where each grid cell is 1.2 degrees longitude wide and 1.2 degrees latitude high:

```
g = Grid('geographic', 1.2)
```

Note that the grid cell size is measured in degrees while the distances between points are measured in kilometers. The rationale for this difference may not be easy to see. Geographic point coordinates are provided in degrees. To find the appropriate grid cell for a single one of these points, the `Grid` instance will need to determine the cell from the two degree values. It is easier to define the grid size in those units. The distance measure remains the `haversineDistance()` function and `Grid` methods that calculate distances will return values in kilometers.

Like the `PointList`, if the `Grid` constructor is passed any string other than `'cartesian'` or `'geographic'`, an exception will be raised, as shown in [Listing 12-14](#).

Listing 12-14 *The Grid Constructor*

```
class Grid(object):
    def __init__(self, kind, cellSize):
        if cellSize <= 0:
            raise Exception("Cell size must be positive")
        self.__cellSize = cellSize

        if kind == 'cartesian':
            self.__distance = euclideanDistance

        elif kind == 'geographic':
            self.__distance = haversineDistance

        else:
```

```
        raise Exception("Invalid kind of coordinate")

    # dict key: (row, col) tuple
    # dict data: list of (a, b, data) tuples
    self.__cells = dict()
```

There are three private attributes: the size of each grid cell (`__cellSize`), a reference to the distance function appropriate to either Cartesian or geographic coordinates, and a reference to an initially empty hash table (dictionary) that will contain the cells of the grid (`__cells`). The constructor initializes all these attributes based on the arguments provided.

Inserting Points

Points are inserted into the `Grid` object using the `insert()` method. From the client's perspective, points are inserted in a fashion identical to the `PointList` insert. The `insert()` method is passed the *x, y* coordinates (or longitude, latitude coordinates) and an associated data item, as shown in [Listing 12-15](#).

Listing 12-15 *The Grid Insert Method*

```
class Grid(object):
...
    # inputs: x,y or long,lat
    # returns row,col tuple specifying grid cell
    def __getCell(self, a, b):
        col = floor(a / self.__cellSize)
        row = floor(b / self.__cellSize)
        return row, col

    # Insert either an x,y,data or longitude,latitude,data point
    def insert(self, a, b, data):
        cell = self.__getCell(a, b) # which cell contains point?
        if cell in self.__cells:    # existing cell?
            c = self.__cells[cell]
            for i in range(len(c)):
                p = c[i] # For each point in cell
                if p[0] == a and p[1] == b: # replace data for
                    c[i] = (a, b, data) # a duplicate
            return

        c.append((a,b,data)) # append new point to cell
```

```
else:
    self.__cells[cell] = [(a,b,data)] # create new cell
```

The private method `__getCell()` determines the row and column of the cell to contain the inserted point by dividing each of the coordinates by the cell size and then taking the floor (that is, the next lowest int) of the result. (The `floor()` function is provided by Python's math module). The cell's row, column coordinates are stored as a tuple in the `cell` variable. Because this could be the first point in the cell, the `insert()` method first checks to see whether a list of points is already associated with the cell using the expression `cell in self.__cells`. That expression tests whether the `__cells` hash table has an entry for the key, `cell`. If nothing is stored yet at that cell, the `else:` clause just creates a new grid cell in the `__cells` dictionary and assigns to it a new list containing only the `(a, b, data)` tuple.

If the cell already exists, it is necessary to determine whether a point with the same `a, b` coordinates has already been inserted in the cell, by looping through all the `(a, b, data)` tuples stored in the cell's list. Like insertions in `PointList`, if a duplicate is found, then the data already stored for that point in the list is replaced. But, if a duplicate is not found, then the new `(a, b, data)` point is appended to the cell's list.

Insertion is accomplished by possibly creating a new entry in the dictionary to contain a new list that is an $O(1)$ operation or by checking an existing cell's list for a duplicate and possibly appending a point to the end of the list which is an $O(N)$ operation. The grid cell contains only a portion of the N items stored in the full structure, but that number is proportional to the total number of items. The resulting complexity for `insert()` is therefore $O(N)$. We discuss this further in the “[Big O and Practical Considerations](#)” and the “[Theoretical Performance and Optimizations](#)” section.

Finding an Exact Match

The client uses the `findExact()` method to search for the point contained in the `Grid` object that exactly matches the specified `x, y` or longitude, latitude coordinates. If an exact match is found, the method returns a reference to the data that was stored for the matching point. The client uses `findExact()` in the same fashion as with the `PointList` class.